AlgoFloat

Ocean Intelligence Platform

AI-Powered ARGO Float Data Explorer — Project Documentation

Natural Language → SQL → Interactive Visualisations

React 18	FastAPI	SQLite	Gemini 2.0 Flash	Plotly.js	Leaflet	Python 3.13	Vite 5

Data Source: ARGO Global Float Array — UCSD
Generated: April 2026

1. Project Overview

AlgoFloat — Ocean Intelligence Platform

AlgoFloat is an AI-powered oceanographic data exploration platform that allows users to query, visualise, and interact with real-world ARGO float data using natural language. The platform uses **Gemini 2.0 Flash** to convert plain English questions into SQL queries, which are executed against a SQLite database of ARGO float observations and returned as interactive charts and tables.

What are ARGO Floats?

The ARGO Program deploys thousands of autonomous profiling floats worldwide. Each float dives to ~2,000 m depth, drifts with ocean currents, then rises while measuring **Temperature**, **Salinity**, and **Pressure** at fixed depth intervals. The data is transmitted via satellite and made freely available, forming the largest real-time global ocean observation network.

System Architecture

The platform follows a modern full-stack architecture with clear separation of concerns:

Layer	Technology	Port	Responsibility
Frontend	React 18 + Vite 5	5173	SPA with Plotly & Leaflet charts
Backend	FastAPI + Uvicorn	8000	REST API, data aggregation, AI integration
Database	SQLite (from CSV)	—	Embedded ocean observation store
AI	Google Gemini 2.0 Flash	cloud	NL → SQL query generation

REST API Endpoints

Method	Path	Description
GET	/api/overview	KPIs, distributions, correlation matrix
GET	/api/map	Float positions with TEMP/PSAL/PRES
GET	/api/trends	Time-sorted observations for trend charts
POST	/api/chat	{ "prompt": "..." } → SQL + results + chart
GET	/api/health	Health check endpoint

2. Dataset Analysis — ARGO Float Observations

2,715	41	27.18	36.91	48.5
Total Records	Unique Floats	Avg Temp (°C)	Avg Salinity (PSU)	Avg Pressure (dbar)

Dataset Columns

The dataset contains 19 columns per observation. Key oceanographic parameters:

Column	Type	Description
PLATFORM_NUMBER	Integer	Unique ARGO float identifier
CYCLE_NUMBER	Integer	Depth profile cycle count
TIME	ISO Text	Observation timestamp (UTC)
LATITUDE	Float	Float position (degrees N)
LONGITUDE	Float	Float position (degrees E)
PRES	Float	Pressure in dbar (~depth in metres)
TEMP	Float	In-situ temperature (°C)
PSAL	Float	Practical salinity (PSU)
TEMP_QC	Char	Temperature quality control flag
PSAL_QC	Char	Salinity quality control flag
PRES_QC	Char	Pressure quality control flag
DATA_MODE	Char	R=realtime, D=delayed-mode, A=adjusted

Descriptive Statistics

Statistic	TEMP	PSAL	PRES
count	2715.0	2715.0	2715.0
mean	27.183	36.911	48.54
std	2.256	0.561	29.712
min	20.891	33.972	0.2
25%	25.459	36.819	22.0
50%	27.768	36.933	48.0
75%	29.308	37.265	74.045
max	30.552	37.714	100.5

Dataset Sample (first 10 rows)

PLATFORM_NUMBER	CYCLE_NUMBER	TIME	LATITUDE	LONGITUDE	TEMP	PSAL	PRES
5906343	161	2025-09-01 02:33:07	22.331	-50.373	28.302	37.395	4.22

PLATFORM_NUMBER	CYCLE_NUMBER	TIME	LATITUDE	LONGITUDE	TEMP	PSAL	PRES
5906343	161	2025-09-01 02:33:07	22.331	-50.373	28.304	37.395	5.82
5906343	161	2025-09-01 02:33:07	22.331	-50.373	28.31	37.395	7.82
5906343	161	2025-09-01 02:33:07	22.331	-50.373	28.312	37.395	10.02
5906343	161	2025-09-01 02:33:07	22.331	-50.373	28.31	37.395	11.52
5906343	161	2025-09-01 02:33:07	22.331	-50.373	28.29	37.485	13.82
5906343	161	2025-09-01 02:33:07	22.331	-50.373	28.209	37.687	15.82
5906343	161	2025-09-01 02:33:07	22.331	-50.373	28.178	37.685	17.82
5906343	161	2025-09-01 02:33:07	22.331	-50.373	28.158	37.684	19.82
5906343	161	2025-09-01 02:33:07	22.331	-50.373	28.148	37.685	21.82

3. Live Website — Output Screenshots

The following screenshots were captured from the live running application (React frontend on port 5173 + FastAPI backend on port 8000) using Playwright at 1440×900 viewport. All data is served in real-time from the ARGO dataset.

3.1 Home Page — Platform Landing

The animated hero section with feature cards showcasing the four main platform capabilities: Dataset Overview, Interactive Map, Parameter Trends, and AI Chatbot.

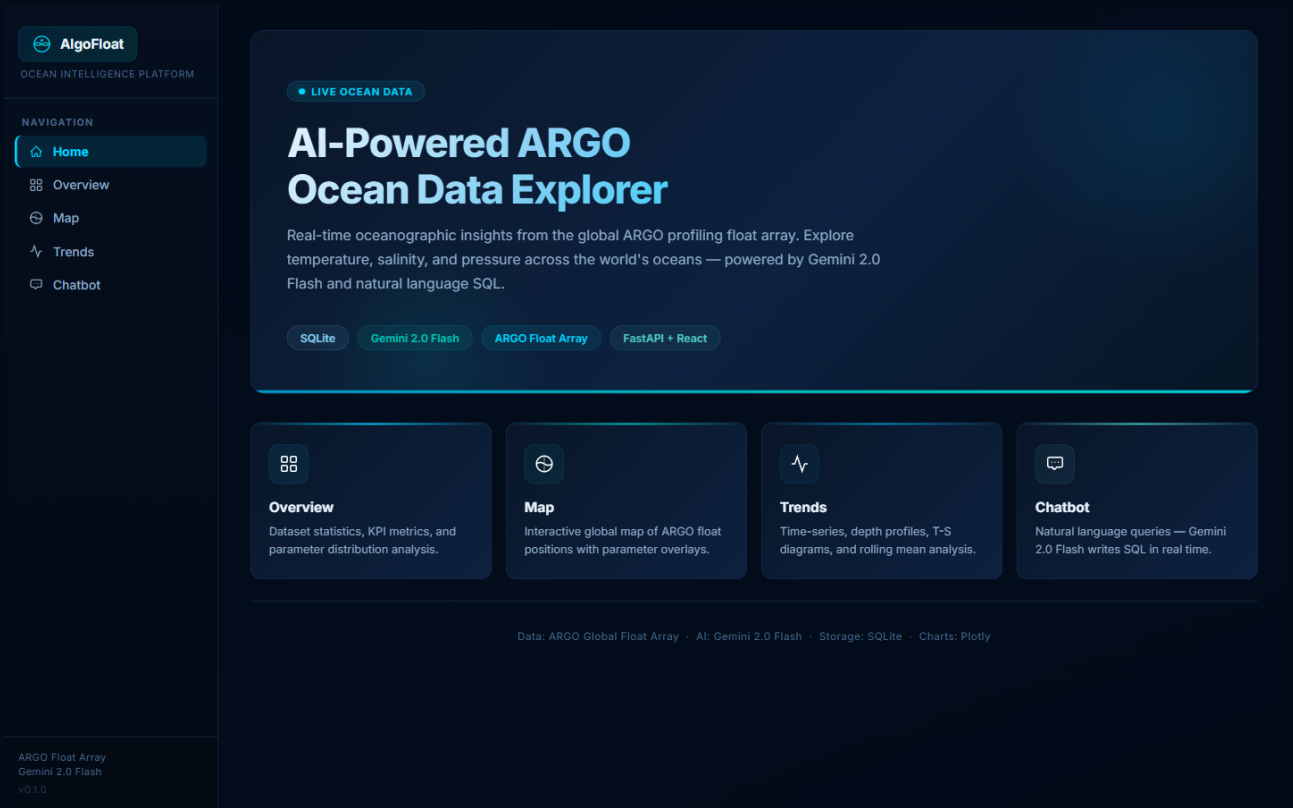

Fig 1 — AlgoFloat Home Page: Dark ocean design with animated hero and feature cards

3.2 Dataset Overview — KPIs, Statistics & Distributions

The Overview page displays 4 real-time KPI metrics (Total Records, Unique Floats, Average Temperature, Average Salinity), a full dataset snapshot table with colour-gradient styling, descriptive statistics, a correlation heatmap (TEMP/PSAL/PRES), a Temperature vs Salinity scatter plot coloured by Pressure, and a per-float records bar chart.

Fig 2 — Overview Page: KPI cards, data table, correlation heatmap, distribution charts

3.3 Global Float Locations Map

The Map page shows all ARGO float positions on a Plotly scattermapbox with a CartoDB Dark Basemap. Markers are colour-coded by Temperature (°C). Platform multi-select filter, geographic distribution histograms (Latitude/Longitude), and a Temperature vs Longitude scatter are shown below the map.

Fig 3 — Map Page: Global ARGO float positions coloured by Temperature with distribution histograms

3.4 Parameter Trends — Time Series & Depth Profiles

The Trends page provides time-series analysis with multi-platform line charts for Temperature over time, salinity depth profiles (TEMP vs PRES, PSAL vs PRES), box plots for statistical distributions, T-S (Temperature-Salinity) diagrams, and rolling mean analysis to reveal seasonal patterns.

Fig 4 — Trends Page: Time-series temperature chart, depth profiles, and rolling mean analysis

3.5 AI Chatbot — Natural Language to SQL

The Chatbot page uses Gemini 2.0 Flash to convert natural language questions into SQLite queries. Results are displayed in an interactive table with CSV download, and auto-charted (time-series or scatter plot depending on the data shape). The schema sidebar shows available columns for reference.

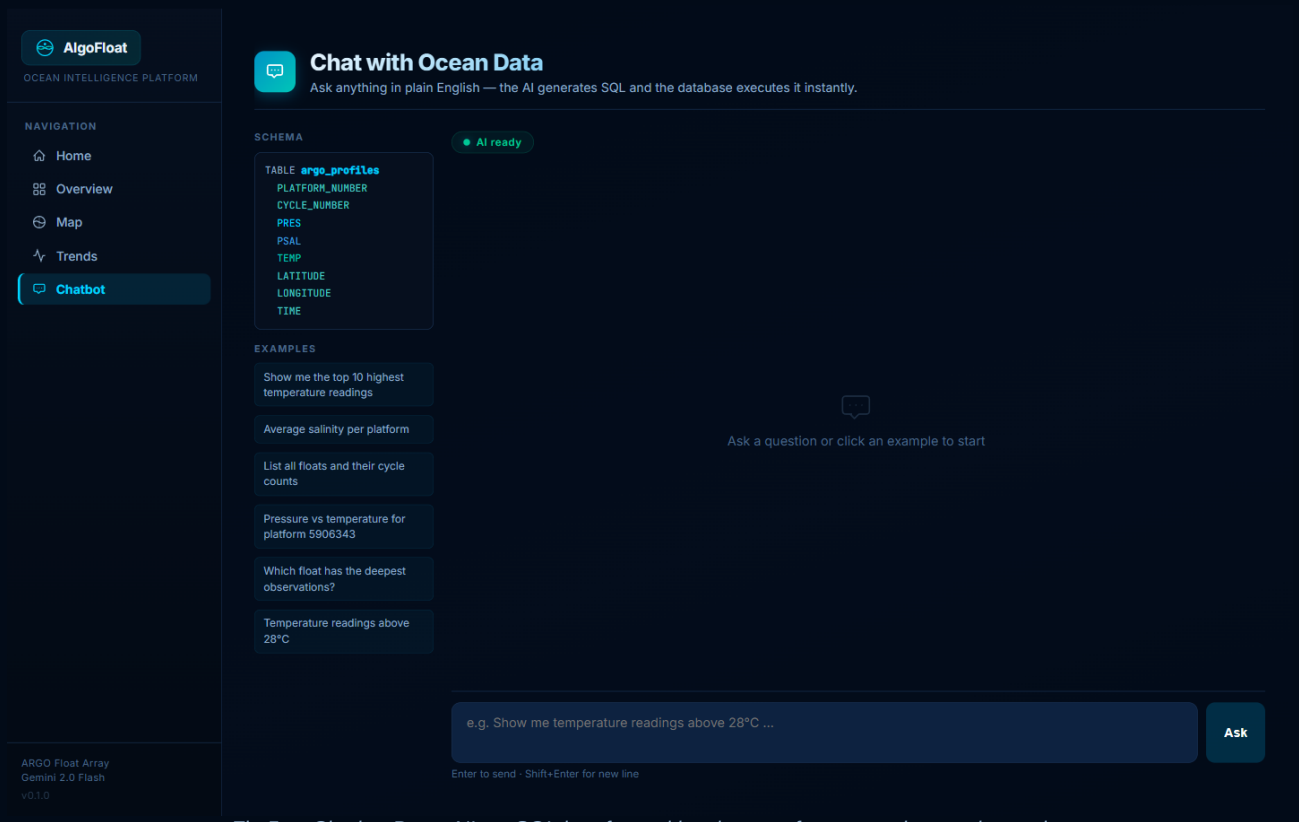

Fig 5 — Chatbot Page: NL → SQL interface with schema reference and example queries

4. Key Source Code

The following listings cover the most critical files in the AlgoFloat codebase. The project uses a clean separation of concerns: FastAPI backend handles all data logic and AI integration, while the React frontend handles rendering and UX.

4.1 backend/main.py — FastAPI Application (REST API)

The central backend file. Defines all five API endpoints. The **/api/overview** endpoint computes KPIs, distributions, and a correlation matrix on-the-fly from CSV. The **/api/chat** endpoint orchestrates the full NL→SQL→Results pipeline.

```
"""
AlgoFloat FastAPI backend.
Serves all data endpoints consumed by the React+Vite frontend.
Run: uvicorn backend.main:app --reload --port 8000
"""

from __future__ import annotations
import os
import sys
import math
import sqlite3

import pandas as pd
from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel

# ■■ Path setup ■■■
```

```
# ■■ Endpoints ■■■
```

```
def get_connection():
if not os.path.exists(DB_PATH):
init_db()
return sqlite3.connect(DB_PATH)


def run_query(query: str) -> pd.DataFrame:
conn = get_connection()
try:
df = pd.read_sql_query(query, conn)
finally:
conn.close()
return df


# Ensure DB is initialised on import
if not os.path.exists(DB_PATH):
init_db()
```

4.3 dashboard/chatbot/gemini_client.py — AI SQL Generator

Wraps the Google Gemini 2.0 Flash API. Sends the user's natural language prompt with a system instruction, then post-processes the response to extract a clean SQL query (stripping markdown fences, junk prefixes, and non-SELECT statements).

```
from google import genai
from google.genai import types
from dotenv import load_dotenv
import os
import re

# Load .env from the dashboard folder (works regardless of cwd)
_ENV_PATH = os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))), ".env")
load_dotenv(_ENV_PATH)

GEMINI_API_KEY = os.getenv("GEMINI_API_KEY", "").strip()
if not GEMINI_API_KEY:
raise ValueError("GEMINI_API_KEY not found. Check dashboard/.env")

_client = genai.Client(api_key=GEMINI_API_KEY)

MODEL_NAME = "gemini-2.0-flash" # Use a Gemini 2.0 model optimized for code generation


def generate_sql(user_prompt: str, system_prompt: str) -> str:
"""
Sends system + user prompt to Gemini and returns a clean SQL query.
"""
response = _client.models.generate_content(
model=MODEL_NAME,
contents=user_prompt,
config=types.GenerateContentConfig(
system_instruction=system_prompt,
),
)
text = response.text.strip()
# Strip markdown code fences if present
text = re.sub(r"```sql\s*", "", text, flags=re.IGNORECASE)
text = re.sub(r"```\s*", "", text)
text = text.strip()
# Strip any garbage prefix before the first SQL keyword
# e.g. Gemini leaks "ite " from "SQLite" at the start
match = re.search(r"\b(SELECT|WITH|INSERT|UPDATE|DELETE)\b", text, re.IGNORECASE)
if match:
text = text[match.start():]
```

```
return text.strip()
```

4.4 dashboard/chatbot/prompt.py — System Prompt

The SQL generation system prompt. Instructs Gemini to output only raw SQL, target the **argo_profiles** table, and restrict to SELECT-only queries.

```
SYSTEM_PROMPT = """
You are an expert SQL query generator.
The database is SQLite. The table is called `argo_profiles` with these columns:
N_POINTS, CYCLE_NUMBER, DATA_MODE, DIRECTION, PLATFORM_NUMBER,
POSITION_QC, PRES, PRES_ERROR, PRES_QC, PSAL, PSAL_ERROR,
PSAL_QC, TEMP, TEMP_ERROR, TEMP_QC, TIME_QC,
LATITUDE, LONGITUDE, TIME

Rules:
- Output ONLY the raw SQL query — no explanation, no markdown, no code fences.
- Start your response directly with SELECT (or WITH for CTEs).
- Only SELECT queries are allowed. No DELETE, UPDATE, INSERT, or DROP.
- Dates in column TIME are ISO text (e.g. '2025-09-01 02:33:07').
"""
```

4.5 frontend/src/App.jsx — React Router & Layout

Sets up BrowserRouter with five routes: /, /overview, /map, /trends, /chatbot. Wraps all pages in a shared Sidebar + main content layout.

```
import { BrowserRouter, Routes, Route } from 'react-router-dom'
import Sidebar from './components/Sidebar'
import Home from './pages/Home'
import Overview from './pages/Overview'
import MapPage from './pages/MapPage'
import Trends from './pages/Trends'
import Chatbot from './pages/Chatbot'

export default function App() {
return (
<BrowserRouter>
<Sidebar />
<main style={{
marginLeft: 'var(--sidebar)',
width: 'calc(100% - var(--sidebar))',
padding: '1.75rem 2.25rem 2.5rem',
minHeight: '100vh',
background: 'linear-gradient(160deg,#020B18 0%,#040D1C 60%,#020B18 100%)',
boxSizing: 'border-box',
}}>
<Routes>
<Route path="/" element={<Home />} />
<Route path="/overview" element={<Overview />} />
<Route path="/map" element={<MapPage />} />
<Route path="/trends" element={<Trends />} />
<Route path="/chatbot" element={<Chatbot />} />
</Routes>
</main>
</BrowserRouter>
)
}
```

4.6 frontend/src/api.js — Axios HTTP Client

Centralised API client. All fetch calls go through this module, which sets the base URL and handles response extraction.

```
import axios from 'axios'

const client = axios.create({ baseURL: '/api', timeout: 30000 })

export async function fetchOverview() { return (await client.get('/overview')).data }
export async function fetchMap() { return (await client.get('/map')).data }
export async function fetchTrends() { return (await client.get('/trends')).data }
export async function postChat(prompt) { return (await client.post('/chat', { prompt })).data }
export async function fetchHealth() { return (await client.get('/health')).data }
```

4.7 frontend/src/plotlyTheme.js — Shared Chart Theme

Provides the **oceanLayout()** function that applies a consistent dark-ocean Plotly theme (navy background, cyan gridlines, white text) across all visualisation pages.

```
/**
* Shared Plotly dark-ocean theme for AlgoFloat.
*/

export const CHART_COLORS = [
'#00D4FF','#0094C6','#00C6B8','#4ECDC4',
'#44A5E0','#7ECEF4','#1A6FA0','#B8E4F9',
]

/**
* Returns a Plotly layout object with the ocean dark theme applied.
```

```
* Spread your own overrides on top.
*/
export function oceanLayout(overrides = {}) {
return {
paper_bgcolor: '#0A1628',
plot_bgcolor: '#0D2240',
font: { family:'Inter, sans-serif', color:'#E8F4FD', size:12 },
colorway: CHART_COLORS,
xaxis: {
gridcolor:'#1A3355', linecolor:'#1A3355',
tickfont:{ color:'#8FACC8' }, title_font:{ color:'#8FACC8' },
},
yaxis: {
gridcolor:'#1A3355', linecolor:'#1A3355',
tickfont:{ color:'#8FACC8' }, title_font:{ color:'#8FACC8' },
},
legend: {
bgcolor:'rgba(10,22,40,0.88)', bordercolor:'#1A3355', borderwidth:1,
font:{ color:'#8FACC8', size:11 },
},
margin: { l:44, r:16, t:44, b:44 },
hoverlabel: {
bgcolor:'#0D2240', bordercolor:'#0094C6',
font:{ color:'#E8F4FD', family:'Inter, sans-serif', size:12 },
},
title: { font:{ color:'#E8F4FD', size:13, family:'Inter, sans-serif' } },
...overrides,
}
}

/** Plotly config — hide the mode bar, enable responsive sizing. */
export const plotConfig = {
displayModeBar: true,
displaylogo: false,
modeBarButtonsToRemove: ['sendDataToCloud','editInChartStudio','lasso2d'],
responsive: true,
}
```

4.8 frontend/src/pages/Overview.jsx — Dashboard Overview Page

Fetches **/api/overview** and renders: MetricCards (with count-up animation), a data snapshot table, three histograms (TEMP/PSAL/PRES), a correlation heatmap using Plotly Heatmap, a Temperature vs Salinity scatter plot, and a per-float bar chart.

```
import { useEffect, useState } from 'react'
import createPlotlyComponent from 'react-plotly.js/factory'
import Plotly from 'plotly.js-dist-min'
import { fetchOverview } from '../api'
import { oceanLayout, CHART_COLORS, plotConfig } from '../plotlyTheme'
import PageHeader from '../components/PageHeader'
import MetricCard from '../components/MetricCard'
import SectionHeader from '../components/SectionHeader'
import Spinner from '../components/Spinner'

const Plot = createPlotlyComponent(Plotly)

const ICON = (
<svg width="22" height="22" fill="none" viewBox="0 0 24 24" stroke="white" strokeWidth="1.8">
<rect x="3" y="3" width="7" height="7" rx="1.5" />
<rect x="14" y="3" width="7" height="7" rx="1.5" />
<rect x="3" y="14" width="7" height="7" rx="1.5" />
<rect x="14" y="14" width="7" height="7" rx="1.5" />
</svg>
)

export default function Overview() {
const [data, setData] = useState(null)
```

```
const [error, setError] = useState(null)

useEffect(() => {
fetchOverview().then(setData).catch(e => setError(e.message))
}, [])

if (error) return <div style={{ color: 'var(--err)', padding: '2rem' }}>Error: {error}</div>
if (!data) return <Spinner label="Loading overview data…" />

const corrCols = ['TEMP', 'PSAL', 'PRES']
const corrZ = corrCols.map(r => corrCols.map(c => data.corr[r]?.[c] ?? 0))

const platLabels = data.platform_summary.map(p => String(p.PLATFORM_NUMBER))
const platRecords = data.platform_summary.map(p => p.records)
const platAvgTemp = data.platform_summary.map(p => p.avg_temp)

return (
<div style={{ animation: 'fadeUp 0.4s ease' }}>
<PageHeader icon={ICON} title="Dataset Overview"
subtitle="Summary statistics, KPI metrics, and parameter distributions across the ARGO dataset." />

{/* KPIs */}
<SectionHeader>Key Metrics</SectionHeader>
<div style={{ display: 'grid', gridTemplateColumns: 'repeat(4,1fr)', gap: '1rem' }}>
<MetricCard label="Total Records" value={data.total_records.toLocaleString()} delay="0s" />
<MetricCard label="Unique Floats" value={data.unique_floats.toLocaleString()} delay="0.08s" />
<MetricCard label="Avg Temp (°C)" value={data.avg_temp} delay="0.16s" />
<MetricCard label="Avg Salinity" value={data.avg_salinity} delay="0.24s" />
</div>

<hr className="divider" />

{/* Dataset sample */}
<SectionHeader>Dataset Snapshot (first 20 rows)</SectionHeader>
<div style={{
overflowX: 'auto', border: '1px solid var(--border)',
borderRadius: 'var(--r)', boxShadow: 'var(--sh)',
}}>
<table style={{
width: '100%', borderCollapse: 'collapse',
fontFamily: "'JetBrains Mono',monospace", fontSize: '0.75rem',
}}>
<thead>
<tr style={{ background: 'var(--surf2)' }}>
{data.sample.length > 0 &&
Object.keys(data.sample[0]).map(col => (
<th key={col} style={{
padding: '0.6rem 0.75rem', textAlign: 'left',
color: 'var(--tx2)', fontWeight: 700,
borderBottom: '1px solid var(--border)',
whiteSpace: 'nowrap', fontSize: '0.68rem',
textTransform: 'uppercase', letterSpacing: '0.06em',
}}>
{col}
</th>
))
}
</tr>
</thead>
<tbody>
{data.sample.map((row, i) => (
<tr key={i} style={{
background: i % 2 === 0 ? 'var(--surface)' : 'var(--surf2)',
transition: 'background 0.15s',
}}
onMouseEnter={e => e.currentTarget.style.background = 'rgba(0,148,198,0.08)'}
onMouseLeave={e => e.currentTarget.style.background = i % 2 === 0 ? 'var(--surface)' : 'var(--surf2)'}
>
```

```
{Object.values(row).map((val, j) => (
<td key={j} style={{
padding: '0.5rem 0.75rem', color: 'var(--tx)',
borderBottom: '1px solid rgba(26,51,85,0.5)',
whiteSpace: 'nowrap',
}}>
{val ?? '—'}
</td>
))}
</tr>
))}
</tbody>
</table>
</div>

<hr className="divider" />

{/* Distributions */}
<SectionHeader>Parameter Distributions</SectionHeader>
<div style={{ display: 'grid', gridTemplateColumns: 'repeat(3,1fr)', gap: '1rem' }}>
{[
{ key: 'temp_values', label: 'Temperature (°C)', color: CHART_COLORS[0] },
{ key: 'psal_values', label: 'Salinity (PSU)', color: CHART_COLORS[1] },
{ key: 'pres_values', label: 'Pressure (dbar)', color: CHART_COLORS[2] },
].map(({ key, label, color }) => (
<Plot
key={key}
data={[{
type: 'histogram', x: data[key], nbinsx: 45,
... (69 more lines)
```

4.9 frontend/src/pages/MapPage.jsx — Global Float Map

Renders an **OceanMap** Leaflet component with CartoDB dark tiles. Circle markers colour-coded by TEMP/PSAL/PRES using a linear interpolation. Platform multi-select filter + geographic distribution histograms.

```
import { useEffect, useState, useMemo } from 'react'
import createPlotlyComponent from 'react-plotly.js/factory'
import Plotly from 'plotly.js-dist-min'
import { fetchMap } from '../api'
import { oceanLayout, CHART_COLORS, plotConfig } from '../plotlyTheme'
import PageHeader from '../components/PageHeader'
import SectionHeader from '../components/SectionHeader'
import MetricCard from '../components/MetricCard'
import Spinner from '../components/Spinner'
import OceanMap from '../components/OceanMap'

const Plot = createPlotlyComponent(Plotly)

const ICON = (
<svg width="22" height="22" fill="none" viewBox="0 0 24 24" stroke="white" strokeWidth="1.8">
<circle cx="12" cy="12" r="9.5" />
<path strokeLinecap="round" d="M2.5 12 Q7 8.5 12 12 Q17 15.5 21.5 12" />
<path strokeLinecap="round" d="M12 2.5v19" opacity="0.35" />
</svg>
)

const PARAM_LABELS = {
TEMP: 'Temperature (°C)',
PSAL: 'Salinity (PSU)',
PRES: 'Pressure (dbar)',
}

export default function MapPage() {
const [rawData, setRawData] = useState(null)
const [error, setError] = useState(null)
const [colorBy, setColorBy] = useState('TEMP')
const [selPlatforms, setSelPlatforms] = useState(null)

useEffect(() => {
fetchMap().then(d => { setRawData(d); setSelPlatforms(null) }).catch(e => setError(e.message))
}, [])

const allPlatforms = useMemo(
() => rawData ? [...new Set(rawData.map(d => d.PLATFORM_NUMBER))].sort() : [],
[rawData]
)

const filtered = useMemo(() => {
if (!rawData) return []
if (!selPlatforms || selPlatforms.length === 0) return rawData
const s = new Set(selPlatforms)
return rawData.filter(d => s.has(d.PLATFORM_NUMBER))
}, [rawData, selPlatforms])

if (error) return <div style={{ color: 'var(--err)', padding: '2rem' }}>Error: {error}</div>
if (!rawData) return <Spinner label="Loading map data…" />

const colorValues = filtered.map(d => d[colorBy])
const avg = arr => arr.length ? arr.reduce((s, v) => s + (v ?? 0), 0) / arr.length : 0

return (
<div style={{ animation: 'fadeUp 0.4s ease' }}>
<PageHeader icon={ICON} title="Float Locations Map"
subtitle="Real-time global map of ARGO profiling float positions. Click any marker for full observation details."
/>

{/* Controls bar */}
<div style={{
display: 'flex', gap: '1rem', alignItems: 'flex-end',
```

```
flexWrap: 'wrap', marginBottom: '1rem',
padding: '0.85rem 1rem',
background: 'var(--surface)', border: '1px solid var(--border)', borderRadius: 'var(--r)',
}}>
<div>
<div style={{ fontSize: '0.68rem', fontWeight: 700, color: 'var(--tx2)', textTransform: 'uppercase',
letterSpacing: '0.08em', marginBottom: '0.35rem' }}>
Colour by
</div>
<div style={{ display: 'flex', gap: '0.4rem' }}>
{Object.entries(PARAM_LABELS).map(([v, l]) => (
<button key={v} onClick={() => setColorBy(v)}
style={{
padding: '0.35rem 0.75rem', borderRadius: 20,
background: colorBy === v ? 'linear-gradient(135deg,var(--a1),var(--a2))' : 'var(--surf2)',
border: `1px solid ${colorBy === v ? 'transparent' : 'var(--border)'}`,
color: colorBy === v ? '#fff' : 'var(--tx2)',
fontSize: '0.8rem', fontWeight: 600, cursor: 'pointer',
transition: 'all 0.15s',
}}>
{v}
</button>
))}
</div>
</div>

<div>
<div style={{ fontSize: '0.68rem', fontWeight: 700, color: 'var(--tx2)', textTransform: 'uppercase',
letterSpacing: '0.08em', marginBottom: '0.35rem' }}>
Filter Platforms
</div>
<select
multiple size={1}
value={selPlatforms?.map(String) ?? ['']}
onChange={e => {
const vals = [...e.target.selectedOptions].map(o => Number(o.value)).filter(Boolean)
setSelPlatforms(vals.length ? vals : null)
}}
style={{
background: '#0D2240', border: '1px solid var(--border)',
borderRadius: 'var(--rs)', color: '#E8F4FD', padding: '0.38rem 0.7rem',
fontSize: '0.82rem', outline: 'none', minWidth: 160, cursor: 'pointer',
}}
>
<option value="" style={{ background:'#0D2240', color:'#E8F4FD' }}>All platforms ({allPlatforms.length})</option>
{allPlatforms.map(p => <option key={p} value={p} style={{ background:'#0D2240', color:'#E8F4FD' }}>{p}</option>)}
</select>
</div>

<div style={{ marginLeft: 'auto', display: 'flex', gap: '0.4rem', alignItems: 'center' }}>
<div style={{
width: 8, height: 8, borderRadius: '50%', background: 'var(--ok)',
animation: 'pulseGlow 2s ease infinite',
}} />
<span style={{ fontSize: '0.78rem', color: 'var(--tx2)' }}>
{filtered.length.toLocaleString()} observations · {new Set(filtered.map(d => d.PLATFORM_NUMBER)).size} floats
</span>
</div>
</div>
... (103 more lines)
```

4.10 frontend/src/pages/Trends.jsx — Parameter Trends

Renders time-series line charts per platform, depth-profile scatter plots (TEMP vs PRES, PSAL vs PRES), box plots, T-S diagrams, and rolling-mean analysis using Plotly.js with the shared ocean theme.

```
import { useEffect, useState, useMemo } from 'react'
import createPlotlyComponent from 'react-plotly.js/factory'
import Plotly from 'plotly.js-dist-min'
import { fetchTrends } from '../api'
import { oceanLayout, CHART_COLORS, plotConfig } from '../plotlyTheme'
import PageHeader from '../components/PageHeader'
import SectionHeader from '../components/SectionHeader'
import MetricCard from '../components/MetricCard'
import Spinner from '../components/Spinner'

const Plot = createPlotlyComponent(Plotly)

const ICON = (
<svg width="22" height="22" fill="none" viewBox="0 0 24 24" stroke="white" strokeWidth="1.8">
<polyline points="22 12 18 12 15 21 9 3 6 12 2 12" />
</svg>
)

const PARAMS = [
{ key: 'TEMP', label: 'Temperature (°C)', color: CHART_COLORS[0] },
{ key: 'PSAL', label: 'Salinity (PSU)', color: CHART_COLORS[1] },
{ key: 'PRES', label: 'Pressure (dbar)', color: CHART_COLORS[2] },
]

export default function Trends() {
const [rawData, setRawData] = useState(null)
const [error, setError] = useState(null)
const [param, setParam] = useState('TEMP')
const [showMarkers, setShowMarkers] = useState(false)
const [selPlatforms, setSelPlatforms] = useState(null)

useEffect(() => {
fetchTrends().then(d => { setRawData(d) }).catch(e => setError(e.message))
}, [])

const allPlatforms = useMemo(
() => rawData ? [...new Set(rawData.map(d => d.PLATFORM_NUMBER))].sort() : [],
[rawData]
)

const filtered = useMemo(() => {
if (!rawData) return []
if (!selPlatforms || selPlatforms.length === 0) return rawData
const s = new Set(selPlatforms)
return rawData.filter(d => s.has(d.PLATFORM_NUMBER))
}, [rawData, selPlatforms])

if (error) return <div style={{ color: 'var(--err)', padding: '2rem' }}>Error: {error}</div>
if (!rawData) return <Spinner label="Loading trends data…" />

const pInfo = PARAMS.find(p => p.key === param)
const pLabel = pInfo.label
const values = filtered.map(d => d[param]).filter(v => v != null)
const mean = values.reduce((s,v)=>s+v,0) / values.length
const std = Math.sqrt(values.reduce((s,v)=>s+(v-mean)**2,0)/values.length)

const activePlatforms = selPlatforms?.length ? selPlatforms : allPlatforms

// ■■ Time series — one trace per platform ■■■
```

```
hovertemplate: `Float ${p}<br>Time: %{x}<br>${param}: %{y:.2f}<extra></extra>`,
}
})

// ■■ Depth profile ■■■
```

```
const ICON = (
<svg width="22" height="22" fill="none" viewBox="0 0 24 24" stroke="white" strokeWidth="1.8">
<path strokeLinecap="round" strokeLinejoin="round"
d="M8 10h.01M12 10h.01M16 10h.01M9 16H5a2 2 0 01-2-2V6a2 2 0 012-2h14a2 2 0 012 2v8a2 2 0 01-2 2h-5l-3 3-3-3z" />
</svg>
)

const EXAMPLES = [
'Show me the top 10 highest temperature readings',
'Average salinity per platform',
'List all floats and their cycle counts',
'Pressure vs temperature for platform 5906343',
'Which float has the deepest observations?',
'Temperature readings above 28°C',
]

function toCsv(data, columns) {
const rows = data.map(r => columns.map(c => JSON.stringify(r[c] ?? '')).join(','))
return [columns.join(','), ...rows].join('\n')
}

function autoChart(data, columns) {
if (!data?.length) return null
if (columns.includes('TIME')) {
const yCols = ['TEMP','PSAL','PRES',...columns.filter(c => typeof data[0][c]===='number' && c!=='TIME')]
const yCol = yCols.find(c => columns.includes(c) && c !== 'TIME')
if (!yCol) return null
const sorted = [...data].sort((a,b)=>a.TIME<b.TIME?-1:1)
const platforms = columns.includes('PLATFORM_NUMBER')
? [...new Set(sorted.map(d=>d.PLATFORM_NUMBER))]
: [null]
return {
traces: platforms.map((p,i) => {
const rows = p!==null ? sorted.filter(d=>d.PLATFORM_NUMBER===p) : sorted
return { type:'scatter', mode:'lines', name: p!==null?String(p):yCol,
x: rows.map(d=>d.TIME), y: rows.map(d=>d[yCol]),
line: { color: CHART_COLORS[i%CHART_COLORS.length], width: 2 } }
}),
layout: oceanLayout({ height:320,
title: { text:`${yCol} over Time`, font:{color:'#E8F4FD',size:13} },
xaxis: { ...oceanLayout().xaxis, title:{text:'Date/Time',font:{color:'#8FACC8'}} },
yaxis: { ...oceanLayout().yaxis, title:{text:yCol,font:{color:'#8FACC8'}} } }),
}
}
const numCols = columns.filter(c => data[0] && typeof data[0][c]===='number')
if (numCols.length >= 2) {
return {
traces: [{ type:'scatter', mode:'markers',
x: data.map(d=>d[numCols[0]]), y: data.map(d=>d[numCols[1]]),
marker: { color: CHART_COLORS[0], opacity: 0.7, size: 6 } }],
layout: oceanLayout({ height:320,
title: { text:`${numCols[0]} vs ${numCols[1]}`, font:{color:'#E8F4FD',size:13} },
xaxis: { ...oceanLayout().xaxis, title:{text:numCols[0],font:{color:'#8FACC8'}} },
yaxis: { ...oceanLayout().yaxis, title:{text:numCols[1],font:{color:'#8FACC8'}} } }),
}
}
return null
}

export default function Chatbot() {
const [messages, setMessages] = useState([])
const [input, setInput] = useState('')
const [loading, setLoading] = useState(false)
const [sqlOpen, setSqlOpen] = useState({})
const bottomRef = useRef(null)

useEffect(() => {
bottomRef.current?.scrollIntoView({ behavior: 'smooth' })
```

```
}, [messages, loading])

async function send() {
const text = input.trim()
if (!text || loading) return
setInput('')
setMessages(prev => [...prev, { role:'user', text }])
setLoading(true)
try {
const res = await postChat(text)
setMessages(prev => [...prev, { role:'assistant', res }])
} catch (e) {
setMessages(prev => [...prev, { role:'assistant', res:{ error: e.message } }])
}
setLoading(false)
}

function handleKey(e) {
if (e.key===='Enter' && !e.shiftKey) { e.preventDefault(); send() }
}

function download(data, columns) {
const a = document.createElement('a')
a.href = URL.createObjectURL(new Blob([toCsv(data, columns)],{type:'text/csv'}))
a.download = 'query_results.csv'; a.click()
}

return (
<div style={{ animation:'fadeUp 0.4s ease', display:'flex', flexDirection:'column', height:'calc(100vh - 4rem)',
overflow:'hidden' }}>
<PageHeader icon={ICON} title="Chat with Ocean Data"
subtitle="Ask anything in plain English — the AI generates SQL and the database executes it instantly." />

<div style={{ flex:1, display:'flex', gap:'1.25rem', overflow:'hidden', minHeight:0 }}>

{/* ■■ Left: schema + examples ■■ */}
<div style={{
width: 200, flexShrink:0, display:'flex', flexDirection:'column', gap:'0.6rem',
overflowY:'auto', paddingBottom:'0.5rem', minHeight:0,
}}>
<div style={{ fontSize:'0.67rem', fontWeight:700, color:'var(--tx3)', textTransform:'uppercase',
letterSpacing:'0.12em' }}>
Schema
</div>
<div style={{
... (249 more lines)
```

4.12 frontend/src/components/OceanMap.jsx — Leaflet Map Component

React-Leaflet component with CartoDB Dark Matter tiles. Computes per-marker colour using linear interpolation between NAVY and CYAN based on normalised parameter value. Includes legend with min/mid/max labels.

```
/**
* OceanMap — Real interactive Leaflet map with CartoDB Dark Matter tiles.
* Renders ARGO float positions as colour-coded circle markers.
*/
import { useEffect, useRef } from 'react'
import { MapContainer, TileLayer, CircleMarker, Tooltip, useMap } from 'react-leaflet'
import 'leaflet/dist/leaflet.css'

const PARAM_LABELS = {
TEMP: 'Temperature (°C)',
PSAL: 'Salinity (PSU)',
PRES: 'Pressure (dbar)',
}

// 5-stop colour scale: deep navy → ocean blue → cyan → teal → warm
const COLOR_STOPS = [
[2, 11, 24], // t=0.00 #020B18
[0, 80, 180], // t=0.25 deep blue
[0, 148, 198], // t=0.50 ocean blue
[0, 198, 184], // t=0.75 teal
[0, 212, 255], // t=1.00 cyan
]

function lerp(a, b, t) { return a + (b - a) * t }

function valueToColor(value, min, max) {
const t = Math.max(0, Math.min(1, (value - min) / (max - min || 1)))
const scaled = t * (COLOR_STOPS.length - 1)
const lo = Math.floor(scaled)
const hi = Math.min(lo + 1, COLOR_STOPS.length - 1)
const frac = scaled - lo
const r = Math.round(lerp(COLOR_STOPS[lo][0], COLOR_STOPS[hi][0], frac))
const g = Math.round(lerp(COLOR_STOPS[lo][1], COLOR_STOPS[hi][1], frac))
const b = Math.round(lerp(COLOR_STOPS[lo][2], COLOR_STOPS[hi][2], frac))
return `rgb(${r},${g},${b})`
}

// Legend gradient stops for display
function colorAt(t) {
const scaled = t * (COLOR_STOPS.length - 1)
const lo = Math.floor(scaled)
const hi = Math.min(lo + 1, COLOR_STOPS.length - 1)
const frac = scaled - lo
const r = Math.round(lerp(COLOR_STOPS[lo][0], COLOR_STOPS[hi][0], frac))
const g = Math.round(lerp(COLOR_STOPS[lo][1], COLOR_STOPS[hi][1], frac))
const b = Math.round(lerp(COLOR_STOPS[lo][2], COLOR_STOPS[hi][2], frac))
return `rgb(${r},${g},${b})`
}

// Flyto centre when data changes
function MapController({ center }) {
const map = useMap()
useEffect(() => { map.setView(center, map.getZoom()) }, [center])
return null
}

export default function OceanMap({ data, colorBy = 'TEMP', height = 560 }) {
if (!data || data.length === 0) return null

const values = data.map(d => d[colorBy]).filter(v => v != null)
const minVal = Math.min(...values)
const maxVal = Math.max(...values)
const midVal = ((minVal + maxVal) / 2).toFixed(2)
```

```
const centerLat = data.reduce((s, d) => s + d.LATITUDE, 0) / data.length
const centerLon = data.reduce((s, d) => s + d.LONGITUDE, 0) / data.length

const LABEL = PARAM_LABELS[colorBy] || colorBy

return (
<div style={{ position: 'relative', borderRadius: 12, overflow: 'hidden',
border: '1px solid #1A3355', boxShadow: '0 4px 24px rgba(0,0,0,0.4)' }}>
<MapContainer
center={[centerLat, centerLon]}
zoom={3}
style={{ height, width: '100%', background: '#020B18' }}
preferCanvas
zoomControl
>
<MapController center={[centerLat, centerLon]} />

<TileLayer
url="https://{s}.basemaps.cartocdn.com/dark_all/{z}/{x}/{y}{r}.png"
attribution='&copy; <a href="https://www.openstreetmap.org/copyright" style="color:#8FACC8">OpenStreetMap</a>
contributors &copy; <a href="https://carto.com/attributions" style="color:#8FACC8">CARTO</a>'
subdomains="abcd"
maxZoom={19}
/>

{data.map((d, i) => {
const val = d[colorBy]
const color = val != null ? valueToColor(val, minVal, maxVal) : '#1A3355'
return (
<CircleMarker
key={i}
center={[d.LATITUDE, d.LONGITUDE]}
radius={5}
pathOptions={{
fillColor: color,
fillOpacity: 0.85,
color: 'rgba(0,0,0,0.3)',
weight: 0.5,
}}
>
<Tooltip sticky>
<div style={{
background: '#0A1628', border: '1px solid #1A3355',
borderRadius: 8, padding: '0.5rem 0.7rem',
fontFamily: 'Inter, sans-serif', fontSize: 12, color: '#E8F4FD',
lineHeight: 1.7, minWidth: 160,
}}>
<div style={{ fontWeight: 700, color: '#00D4FF', marginBottom: 4 }}>
Float {d.PLATFORM_NUMBER}
</div>
<div style={{ color: '#8FACC8' }}>Cycle: {d.CYCLE_NUMBER}</div>
<div style={{ color: '#8FACC8' }}>Time: {String(d.TIME).slice(0, 16)}</div>
<hr style={{ border: 'none', borderTop: '1px solid #1A3355', margin: '4px 0' }} />
<div>Temp: <span style={{ color: '#00D4FF', fontWeight: 600 }}>{d.TEMP?.toFixed(2)} °C</span></div>
<div>Salinity: <span style={{ color: '#00C6B8', fontWeight: 600 }}>{d.PSAL?.toFixed(3)}</span></div>
<div>Pressure: <span style={{ color: '#7ECEF4', fontWeight: 600 }}>{d.PRES?.toFixed(1)} dbar</span></div>
<div style={{ color: '#8FACC8', marginTop: 2 }}>
... (34 more lines)
```

4.13 frontend/src/components/MetricCard.jsx — Animated KPI Card

Displays a KPI metric with count-up animation (useEffect + requestAnimationFrame). Used on the Overview and other pages for Total Records, Unique Floats, etc.

```
import { useEffect, useState, useRef } from 'react'

function useCountUp(target, duration = 900) {
```

```
const [display, setDisplay] = useState(null)
const rafRef = useRef(null)

useEffect(() => {
const numeric = parseFloat(String(target).replace(/,/g, ''))
if (isNaN(numeric)) { setDisplay(target); return }

const start = performance.now()
function tick(now) {
const elapsed = now - start
const progress = Math.min(1, elapsed / duration)
const eased = 1 - Math.pow(1 - progress, 3) // ease-out cubic
const current = numeric * eased
// format like the original: if it was an int show int, else keep decimals
const isInt = Number.isInteger(numeric)
setDisplay(isInt
? Math.round(current).toLocaleString()
: current.toFixed(String(target).includes('.') ? String(target).split('.')[1].length : 2)
)
if (progress < 1) rafRef.current = requestAnimationFrame(tick)
}
rafRef.current = requestAnimationFrame(tick)
return () => cancelAnimationFrame(rafRef.current)
}, [target, duration])

return display ?? target
}

export default function MetricCard({ label, value, sub, delay = '0s', color = 'var(--a3)' }) {
const displayed = useCountUp(value)

return (
<div
style={{
background: 'linear-gradient(135deg,var(--surface),var(--surf2))',
border: '1px solid var(--border)',
borderRadius: 'var(--r)',
padding: '1.25rem 1.25rem 1rem',
position: 'relative',
overflow: 'hidden',
boxShadow: 'var(--sh)',
transition: 'transform 0.2s ease, box-shadow 0.2s ease, border-color 0.2s',
animation: `fadeUp 0.4s ease ${delay} both`,
cursor: 'default',
}}
onMouseEnter={e => {
e.currentTarget.style.transform = 'translateY(-4px)'
e.currentTarget.style.boxShadow = 'var(--sh2)'
e.currentTarget.style.borderColor = 'rgba(0,148,198,0.4)'
}}
onMouseLeave={e => {
e.currentTarget.style.transform = ''
e.currentTarget.style.boxShadow = 'var(--sh)'
e.currentTarget.style.borderColor = 'var(--border)'
}}
>
{/* top gradient bar */}
<div style={{
position: 'absolute', top: 0, left: 0, right: 0, height: 2,
background: 'linear-gradient(90deg,var(--a1),var(--a2),var(--a3))',
}} />
{/* ambient glow */}
<div style={{
position: 'absolute', top: -20, right: -20, width: 80, height: 80,
background: `radial-gradient(circle, ${color}18 0%, transparent 70%)`,
borderRadius: '50%', pointerEvents: 'none',
}} />
```

```
<div style={{
fontSize: '0.7rem', fontWeight: 700, color: 'var(--tx2)',
textTransform: 'uppercase', letterSpacing: '0.1em', marginBottom: '0.6rem',
}}>
{label}
</div>
<div style={{
fontSize: '2rem', fontWeight: 800, color, lineHeight: 1,
fontVariantNumeric: 'tabular-nums',
... (12 more lines)
```

4.14 pyproject.toml — Python Project Configuration

Declares all Python dependencies and the uv console scripts: **uv run start** (Streamlit), **uv run dev** (Streamlit dev), **uv run serve** (FastAPI).

```
[project]
name = "algofloat"
version = "0.1.0"
description = "AI-powered ARGO ocean float data dashboard"
requires-python = ">=3.10"
dependencies = [
"streamlit>=1.35.0",
"pandas>=2.0.0",
"plotly>=5.0.0",
"google-genai>=1.0.0",
"langgraph>=0.2.0",
"python-dotenv>=1.0.0",
"fastapi>=0.135.2",
"uvicorn[standard]>=0.42.0",
]

[project.scripts]
start = "algofloat.cli:start"
dev = "algofloat.cli:dev"
serve = "algofloat.cli:serve"

[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[tool.hatch.build.targets.wheel]
packages = ["algofloat"]
```

4.15 frontend/package.json — Node.js Dependencies

Defines npm dependencies: React, Vite, Plotly.js, React-Leaflet, Axios, and React-Router-DOM. Build scripts: **npm run dev** (Vite dev server), **npm run build** (production bundle).

```
{
"name": "algofloat-frontend",
"version": "0.1.0",
"private": true,
"type": "module",
"scripts": {
"dev": "vite",
"build": "vite build",
"preview": "vite preview"
},
"dependencies": {
"axios": "^1.7.2",
"leaflet": "^1.9.4",
"plotly.js-dist-min": "^2.33.0",
"react": "^18.3.1",
"react-dom": "^18.3.1",
"react-leaflet": "^4.2.1",
"react-plotly.js": "^2.6.0",
"react-router-dom": "^6.23.1"
},
"devDependencies": {
"@vitejs/plugin-react": "^4.3.1",
"vite": "^5.3.1"
}
}
```

5. Design System — Dark Ocean Theme

AlgoFloat uses a custom dark-ocean design system defined entirely in CSS custom properties and applied consistently across all components and Plotly charts.

Token	Value	Swatch	Usage
`--bg`	`#020B18`		Page background — deep navy
`--surface`	`#0A1628`		Card background
`--surf2`	`#0D2240`		Secondary surface
`--a1`	`#0094C6`		Ocean blue accent
`--a2`	`#00C6B8`		Teal accent
`--a3`	`#00D4FF`		Cyan accent — primary interactive
`--tx`	`#E8F4FD`		Primary text — light cyan
`--tx2`	`#8FACC8`		Secondary text — muted blue
`--border`	`#1A3355`		Border colour
`--ok`	`#00C896`		Success green
`--warn`	`#FFB347`		Warning orange
`--err`	`#FF6B6B`		Error red

CSS Keyframe Animations

Animation	Duration	Purpose
`fadeUp`	0.4s	Slide up + fade in — used on page load for cards
`fadeIn`	0.4s	Opacity only — used on hero text
`pulseGlow`	1.8s	Box-shadow pulse — used on active nav items
`gradientShift`	3.5s	Background-position animation — hero gradient
`spin`	0.7s	Rotation — loading spinner

6. Application Pages Summary

Page	Route	Key Features
Home	`/`	Animated hero, feature cards, tech-stack badges, call-to-action
Overview	`/overview`	4 KPI metrics, dataset snapshot table, histograms (TEMP/PSAL/PRES), correlation heatmap, T-S scatter, per-float bar chart
Map	`/map`	Global Leaflet map with CartoDB Dark tiles, temperature-coloured markers, platform filter, lat/lon distribution histograms, T-S scatter
Trends	`/trends`	Time-series line charts per float, depth-profile scatter (TEMP vs PRES), box plots, T-S diagram, rolling-mean analysis
Chatbot	`/chatbot`	Natural language input → Gemini SQL → SQLite → results table + CSV download + auto-chart (time-series or scatter)

How to Run the Application

Step 1 — Set API Key

```
Create dashboard/.env and add: GEMINI_API_KEY=your_key_here
```

Step 2 — Start Backend (Terminal 1)

```
cd AlgoFloat-main
uv run serve
→ FastAPI on http://localhost:8000
```

Step 3 — Start Frontend (Terminal 2)

```
cd AlgoFloat-main/frontend
npm run dev
→ React on http://localhost:5173
```

Step 4 — Open Browser

```
Navigate to http://localhost:5173
```

Prerequisites

Requirement	Version	Purpose
uv	0.9+	Python package manager and script runner
Python	3.10+	Backend runtime
Node.js	18+	Frontend dev server and build tool
npm	9+	JavaScript package manager
Gemini API Key	—	Google AI Studio — free tier available

7. Data Flow & AI Pipeline

7.1 Standard Data Flow (Chart Pages)

```
User Browser (React)
■ HTTP GET /api/overview (or /map, /trends)
▼
FastAPI Backend (port 8000)
■ pd.read_csv("data/sample_argo.csv")
■ Compute aggregations (groupby, describe, corr)
■ _clean() — strip NaN/Inf for JSON serialisation
▼
JSON Response → React page → Plotly.js / Leaflet charts
```

7.2 AI Chatbot Pipeline (NL → SQL → Results)

```
User types: "Show me the top 10 highest temperature readings"
■
▼ POST /api/chat { "prompt": "..." }
FastAPI (backend/main.py — chat endpoint)
■
▼ generate_sql(prompt, SYSTEM_PROMPT)
Gemini 2.0 Flash (Google AI Cloud)
■ System prompt: output only raw SQL, SELECT only
■ Response: "SELECT * FROM argo_profiles ORDER BY TEMP DESC LIMIT 10"
■
▼ Post-processing (gemini_client.py)
1. Strip markdown fences (```sql ... ```)
2. Strip junk prefix (e.g. "ite " leaked from "SQLite")
3. Find first SQL keyword (SELECT / WITH)
■
▼ run_query(sql) → SQLite argo_profiles table
pd.read_sql_query() returns DataFrame
■
▼ JSON Response
{ sql, data: [...], columns: [...], row_count: N, error: null }
■
▼ React Chatbot.jsx
Renders table + auto-chart:
• TIME column present → Plotly line chart (time-series per platform)
• 2+ numeric columns → Plotly scatter plot
• Otherwise → Table only
```

7.3 Database Initialisation Flow

```
On first import of dashboard/db.py:
if not os.path.exists("data/argo.db"):
init_db()
↓
pd.read_csv("data/sample_argo.csv")
↓
df.to_sql("argo_profiles", sqlite3.connect("data/argo.db"),
if_exists="replace", index=False)

Result: argo.db with table argo_profiles (~2,715 rows, 19 columns)
```

8. Frontend Styling — CSS Design Tokens & Global Styles

The global CSS file defines all design tokens as CSS custom properties, keyframe animations, base element resets, layout primitives (sidebar + main), card styles, button styles, input styles, and table styles.

```
/* ═══════════════════════════════════════════
AlgoFloat — Global Styles & Design Tokens
Ocean Intelligence Platform
═══════════════════════════════════════════ */

:root {
--bg: #020B18;
--surface: #0A1628;
--surf2: #0D2240;
--a1: #0094C6;
--a2: #00C6B8;
--a3: #00D4FF;
--tx: #E8F4FD;
--tx2: #8FACC8;
--tx3: #4A6888;
--border: #1A3355;
--ok: #00C896;
--warn: #FFB347;
--err: #FF6B6B;
--r: 12px;
--rs: 8px;
--sh: 0 4px 24px rgba(0,148,198,0.10);
--sh2: 0 8px 32px rgba(0,148,198,0.22);
--sidebar: 240px;
}

*,*::before,*::after { box-sizing:border-box; margin:0; padding:0; }

html,body {
font-family:'Inter',sans-serif;
background:var(--bg);
color:var(--tx);
min-height:100vh;
-webkit-font-smoothing:antialiased;
}

#root { min-height:100vh; }

a { color:inherit; text-decoration:none; }

/* ══ Force dark select/option everywhere ══ */
select {
background:#0D2240 !important;
color:#E8F4FD !important;
border-color:var(--border) !important;
color-scheme: dark;
}
option {
background:#0D2240 !important;
color:#E8F4FD !important;
}

/* ══ Scrollbar ══ */
::-webkit-scrollbar { width:5px; height:5px; }
::-webkit-scrollbar-track { background:var(--bg); }
::-webkit-scrollbar-thumb { background:var(--border); border-radius:3px; }
::-webkit-scrollbar-thumb:hover { background:var(--a1); }

/* ══ Keyframes ══ */
@keyframes fadeUp {
from { opacity:0; transform:translateY(12px); }
to { opacity:1; transform:translateY(0); }
```

```
}
@keyframes fadeIn {
from { opacity:0; }
to { opacity:1; }
}
@keyframes gradientShift {
0%,100% { background-position:0% 50%; }
50% { background-position:100% 50%; }
}
@keyframes pulseGlow {
0%,100% { box-shadow:0 0 8px rgba(0,212,255,0.3); }
50% { box-shadow:0 0 22px rgba(0,212,255,0.65); }
}
@keyframes spin {
to { transform:rotate(360deg); }
}
@keyframes blink {
0%,100% { opacity:1; }
50% { opacity:0; }
}
@keyframes shimmer {
0% { background-position:-200% center; }
100% { background-position: 200% center; }
}

/* ■■ Utility classes ■■ */
.fade-up { animation:fadeUp 0.4s ease both; }
.fade-in { animation:fadeIn 0.4s ease both; }

.divider {
border:none;
border-top:1px solid var(--border);
margin:1.5rem 0;
}

/* ■■ Leaflet overrides — force dark theme ■■ */
.leaflet-container {
background:#020B18 !important;
... (24 more lines)
```

Project Summary

AlgoFloat successfully demonstrates a full end-to-end AI-powered data science web application. The platform combines:

■ Real-time Oceanographic Data

2,715 observations from 41 ARGO floats measuring Temperature (20.9°C – 30.6°C), Salinity (34.0 – 37.7 PSU), and Pressure across the global ocean.

■ AI-Driven Querying

Google Gemini 2.0 Flash converts natural language questions into precise SQL queries, making oceanographic data accessible to non-technical users.

■ Modern Full-Stack Architecture

React 18 + Vite frontend with Plotly.js and Leaflet maps, FastAPI backend with clean REST API design, and SQLite embedded database.

■ Interactive Visualisations

5 rich pages: global interactive map, time-series trends, depth profiles, T-S diagrams, correlation heatmaps, and auto-charting chatbot results.

■ Dark Ocean Design System

A cohesive dark-blue/cyan colour palette with smooth CSS animations creating an immersive oceanographic aesthetic throughout the platform.

AlgoFloat — Ocean Intelligence Platform

Data source: Argo Program — UCSD (argo.ucsd.edu) | Powered by Google Gemini 2.0 Flash